Smart Campus Transport System

VITyarthi – Build Your Own Project

Integrated M.Tech CSE (Cyber Security)
VIT Bhopal University

Final Project Report

1. Introduction

The Smart Campus Transport System is a console-based Python application for managing campus transport records. It provides a structured workflow for registering students, managing buses and routes, creating bookings, searching students and producing transport reports. JSON is used for persistent local storage.

2. Problem Statement

Manual campus transport management can make student, bus, route and booking records difficult to maintain. The proposed system centralizes these records in a simple menu-driven application and validates important inputs before saving them.

3. Objectives

• Identify a practical campus transport management problem.

• Design a modular technical solution using Python and JSON.

• Provide clear input/output and menu-driven workflow.

• Apply validation, error handling, persistence and automated testing.

• Document the system using architecture, workflow and UML-style diagrams.

4. Functional Requirements

• Student management: register and search students; prevent duplicate roll numbers.

• Bus management: add and view buses; validate positive capacity and duplicate bus numbers.

• Route management: add and view routes; ensure each route references an existing bus.

• Booking: validate student/bus existence, prevent duplicate bookings and enforce bus capacity.

• Reporting: display counts and detailed transport records.

• Storage: load and save students, buses, routes and bookings using data.json.

5. Non-Functional Requirements

• Usability – simple terminal menu and clear prompts/messages.

• Reliability – validation and controlled handling of missing/invalid JSON data.

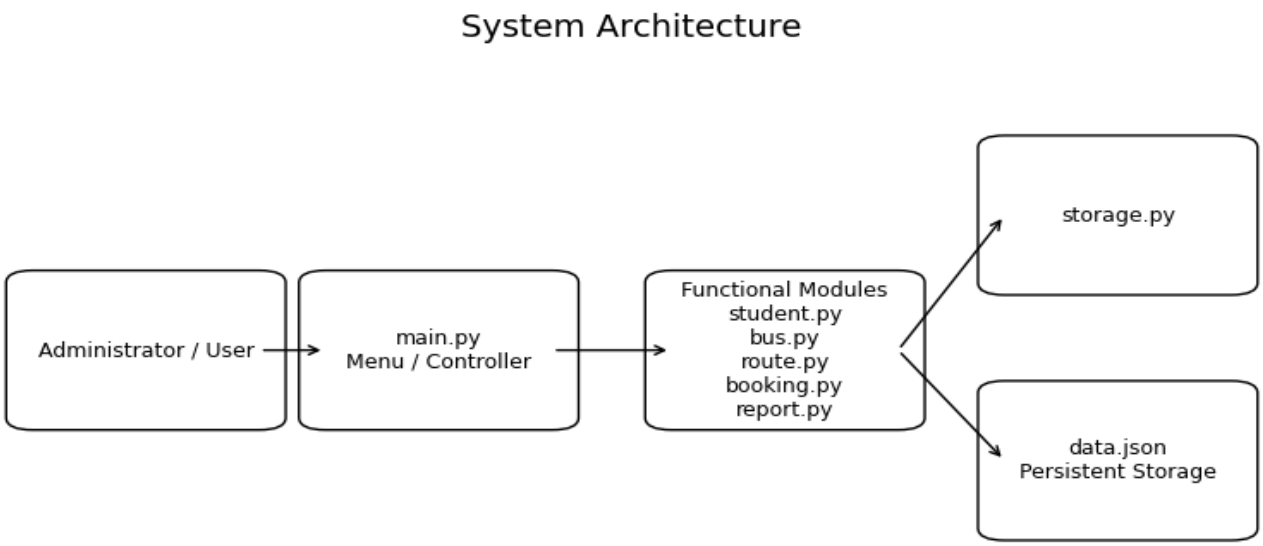
• Maintainability – separate Python modules for each major responsibility.

• Performance – in-memory list processing is appropriate for a small campus dataset.

• Resource efficiency – standard-library JSON persistence avoids a database server.

• Error handling – invalid fields, duplicate records, missing entities and full buses are rejected.

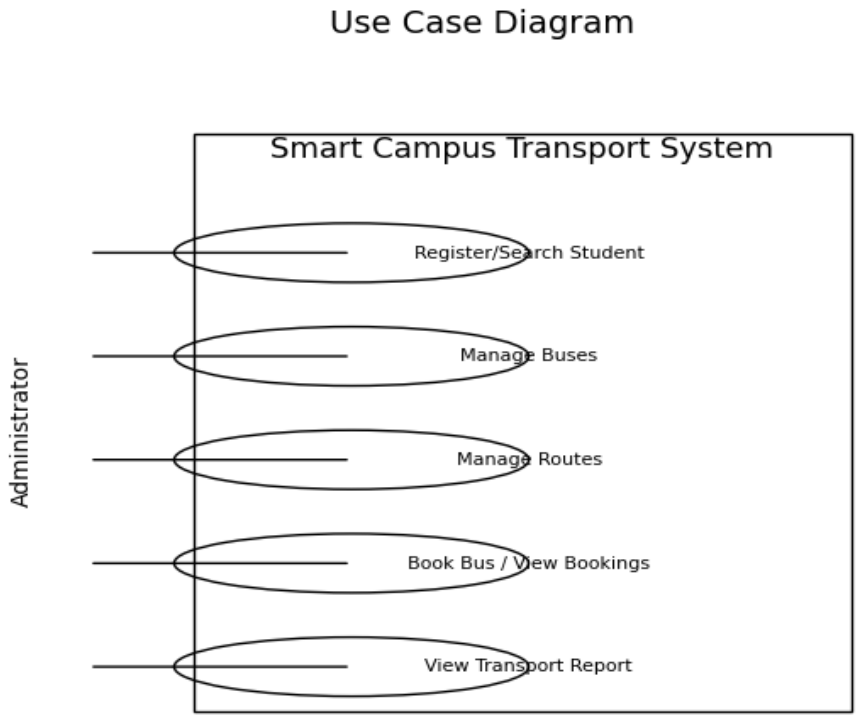
6. System Architecture



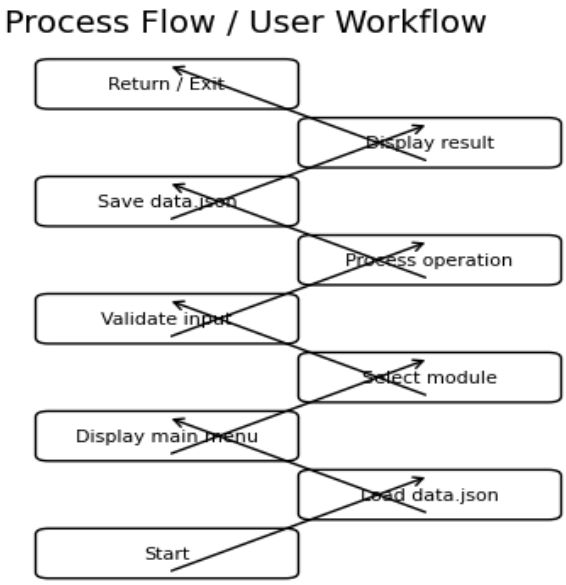
The application uses main.py as the menu/controller layer. Functional modules handle students, buses, routes, bookings and reports, while storage.py provides persistence to data.json.

7. Design Diagrams

7.1 Use Case Diagram

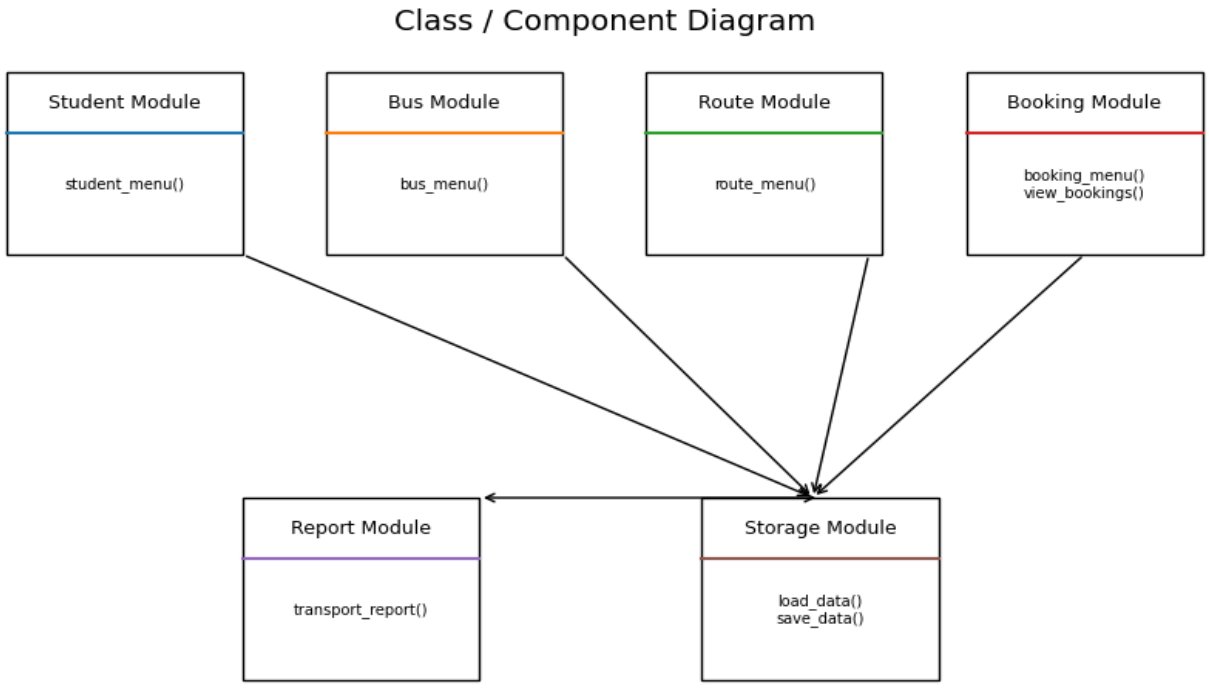


7.2 Workflow Diagram

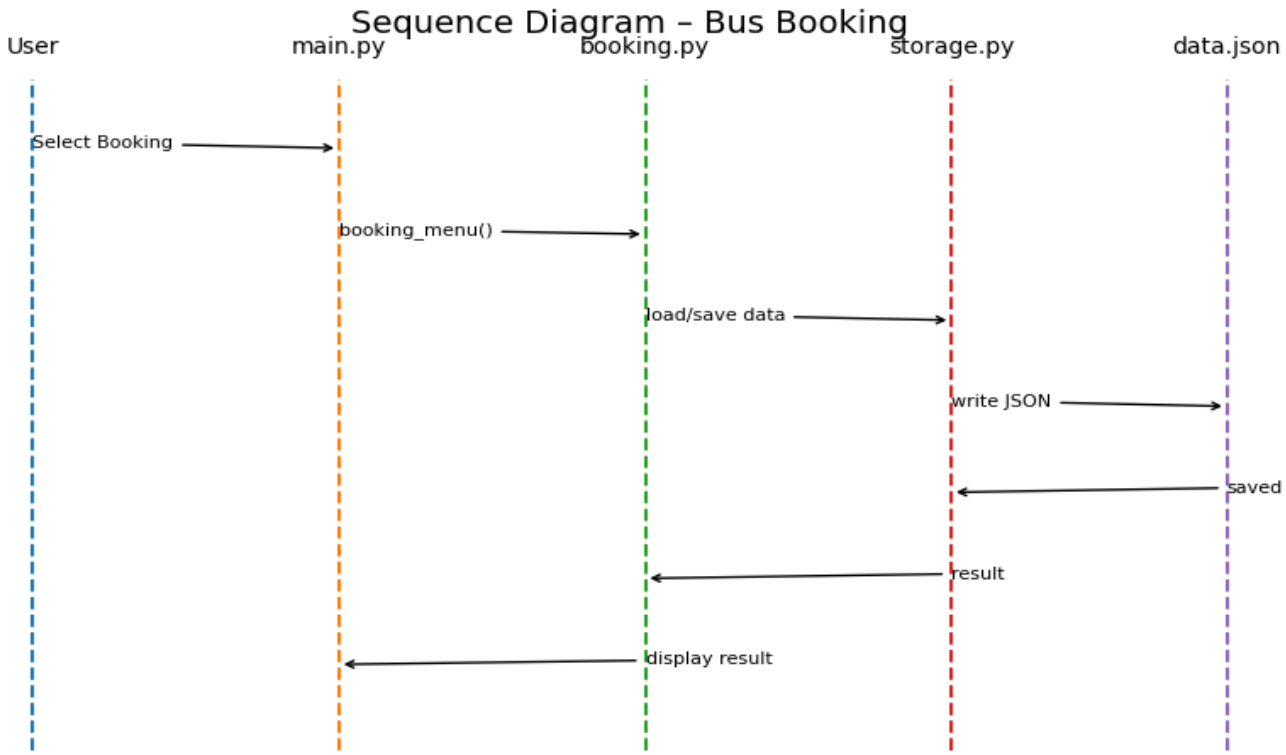


The workflow starts by loading data, presenting the main menu, validating the selected operation, processing it, persisting changes and returning the result to the user.

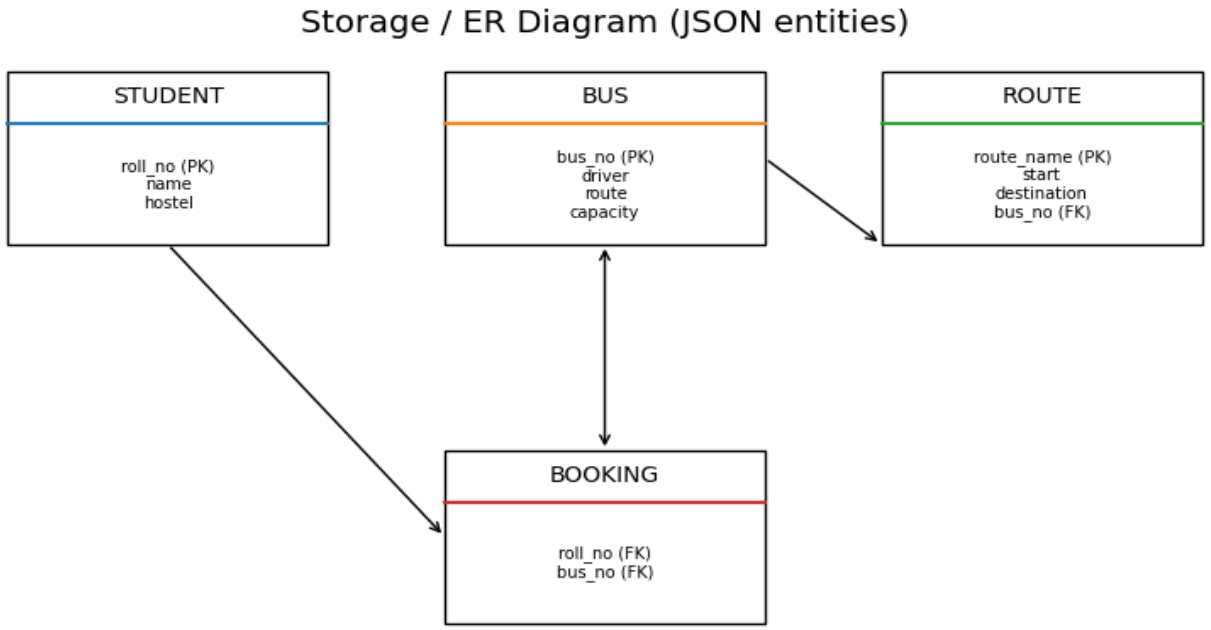
7.3 Class / Component Diagram



7.4 Sequence Diagram



7.5 ER / Storage Design



8. Design Decisions & Rationale

• Python was selected for straightforward procedural/modular implementation.

• JSON was selected because the project is small and requires human-readable local persistence.

• Separate modules reduce coupling and make testing and maintenance easier.

• Capacity enforcement prevents bookings from exceeding the configured bus capacity.

• Validation helpers centralize reusable checks for required text and positive integer values.

9. Implementation Details

Modules: main.py, student.py, bus.py, route.py, booking.py, report.py, storage.py and validation.py. The tests directory contains automated validation tests. The program uses dictionaries/lists for records and JSON serialization for persistence.

10. Screenshots / Results

```
Smart Campus Transport System - Automated Testing
================================================
Command: python -m pytest -q

8 passed in 0.04s

Covered checks:
✓ JSON data file and schema
✓ Route references existing bus
✓ Positive integer bus capacity
✓ Full-bus booking rejection
✓ Available-seat booking
✓ Required-field validation
✓ Positive-integer validation
```

The final automated test suite completed successfully with 8 passing tests.

11. Testing Approach

• Data file existence and schema validation.

• Route-to-bus referential validation.

• Positive bus capacity validation.

• Booking rejection when a bus is full.

• Booking success when a seat is available.

• Required text and positive-integer helper tests.

• Python compilation and manual menu-flow checks.

12. Challenges Faced

The main implementation challenges were maintaining consistent JSON records, validating user-entered data, preventing duplicate bookings and enforcing capacity while keeping the application simple and modular.

13. Learnings & Key Takeaways

• Modular Python design improves readability and maintainability.

• Input validation is essential for reliable console applications.

• Persistent JSON storage can be effective for small structured datasets.

• Automated tests provide repeatable evidence that important rules work.

• Documentation and diagrams make technical design easier to communicate.

14. Future Enhancements

• Edit and delete operations for records.

• Authentication and role-based access.

• Graphical or web interface.

• GPS/live bus tracking.

• Database-backed storage for larger datasets.

• Advanced analytics such as route utilization and occupancy trends.

15. References

1. VITyarthi – Build Your Own Project: General Project Instructions & Submission Guidelines (provided course document).

2. Python Standard Library documentation concepts used by the project: json, pathlib and command-line execution.

3. Pytest testing framework documentation concepts used for automated tests.

Appendix A – Repository Contents

The final repository contains source modules, data.json, README.md, statement.md, requirements.txt, .gitignore, automated tests and the final project report. The structure satisfies the course requirement for a modular implementation with 5–10 meaningful modules/files and testing.